Spec-driven testing (plan → generate → heal)

End-to-end workflow for authoring and maintaining Playwright tests using `playwright-cli`. The three sections below can be used independently:

- **Planning** — explore the app, produce a spec file describing what to test.
- **Generate** — turn a spec into Playwright test files. Update the spec if it's vague or stale.
- **Heal** — diagnose failing tests, fix the code, reconcile the spec with reality.

All three lean on the same mechanic: run `npx playwright test --debug=cli` in the background, then `playwright-cli attach tw-XXXX` to drive the paused page interactively. See playwright-tests.md for the debug/attach mechanics and `test-generation.md` for how every `playwright-cli` action emits Playwright TypeScript.

---

1. Planning

Goal: produce a spec file (e.g. `specs/<feature>.plan.md`) that enumerates the scenarios to test. **Always** write the spec to a file.

1.1 Prerequisite: workspace

Check the workspace has Playwright installed before anything else:

```
# Either of these confirms a workspace:
test -f playwright.config.ts || test -f playwright.config.js
npx --no-install playwright --version
```

If there is no Playwright install, bootstrap one and let the user pick the defaults:

```
npm init playwright@latest
```

1.2 Prerequisite: seed test

A **seed test** is a minimal test that lands the page in the state every scenario starts from: navigation to the app, any required login, feature flags, etc. Scenarios assume a fresh start *after* the seed. `--debug=cli` pauses *inside* this test, so the seed is where every planning and generation session begins.

Minimum viable seed:

```ts
// tests/seed.spec.ts
import { test } from '@playwright/test';

test('seed', async ({ page }) => {
  await page.goto('https://example.com/');
});
```

Preferred — push navigation into a fixture so scenario tests reuse it:

```ts
// tests/fixtures.ts
import { test as baseTest } from '@playwright/test';
export { expect } from '@playwright/test';

export const test = baseTest.extend({
  page: async ({ page }, use) => {
    await page.goto('https://example.com/');
    await use(page);
  },
});
```

```ts
// tests/seed.spec.ts
import { test } from './fixtures';

test('seed', async ({ page }) => {
  // Fixture already navigates. This empty body tells agents where to start.
});
```

If no seed exists, create one that at least navigates to the app.

1.3 Explore the app

Launch the app via the seed in the background and attach:

```
PLAYWRIGHT_HTML_OPEN=never npx playwright test tests/seed.spec.ts --debug=cli
# wait for "Debugging Instructions" and the session name tw-XXXX
playwright-cli attach tw-XXXX
```

Resume so the seed runs, then probe the app:

```
playwright-cli resume                   # resume so that seed test runs fully
playwright-cli snapshot                 # inventory of interactive elements
playwright-cli click e5                 # follow a flow
playwright-cli eval "location.href"     # read URL / state
playwright-cli show --annotate          # ask the user to point at something
```

Map out:

- Interactive surfaces (forms, buttons, lists, filters, modals).

- Primary user journeys end-to-end.
- Edge cases: empty states, validation errors, very long input, boundary values.
- Persistence: reload, local/session storage, URL fragments.
- Navigation: which controls change the URL, back/forward behaviour.

Important: Do not just open the app url with playwright-cli, always go through the test to capture any custom setup done there. **Important**: Stop the background test when done exploring.

1.4 Write the spec file

Save under `specs/<feature>.plan.md`. Use this structure:

```
# <Feature> Test Plan

## Application Overview

<One paragraph describing what the feature does and why it matters.>

## Test Scenarios

### 1. <Group Name>

**Seed:** `tests/seed.spec.ts`

#### 1.1. <kebab-case-scenario-name>

**File:** `tests/<group>/<kebab-case-scenario-name>.spec.ts`

**Steps:**
  1. <Concrete user step>
     - expect: <observable outcome>
     - expect: <another observable outcome>
  2. <Next step>
     - expect: <outcome>

#### 1.2. <next-scenario>
...

### 2. <Next Group>

**Seed:** `tests/seed.spec.ts`
...
```

Guidelines:

- Each scenario is independent and starts from the seed's fresh state — never chain scenarios.
- Scenario names are kebab-case and match the test file name (`should-add-single-todo` → `should-add-single-todo.spec.ts`).
- Cover happy path, edge cases, validation, negative flows, persistence.
- Write steps at the user level ("Type 'Buy milk' into the input"), not the API level ("call `fill`").
- Put observable outcomes in `- expect:` bullets; each becomes an assertion during generation.

---

2. Generate

Goal: take a spec file and produce Playwright test files. Optionally update the spec if it has drifted.

2.1 Inputs

- **Spec file**, e.g. `specs/basic-operations.plan.md`.
- **Target**: either a single scenario (e.g. `1.2`), a whole group (`1`), or all.
- **Seed file**, read from the `**Seed:**` line of the scenario's group.

2.2 Generate one scenario

For each target scenario, in sequence (never in parallel — scenarios share the seed session):

```
PLAYWRIGHT_HTML_OPEN=never npx playwright test <seed-file> --debug=cli   # backg
playwright-cli attach tw-XXXX
# resume
```

Do not just open the app url with playwright-cli, always go through the test to capture any custom setup done there.

Walk the scenario's `Steps:` one by one with `playwright-cli`, treating the spec as the plan and the live app as the source of truth. If a step is vague ("click the button" — which button?), references an element that no longer exists, or contradicts the app's actual behaviour, use your judgement: update the spec to match what the app really does, then keep going. Editing the spec mid-generation is expected.

Every action prints the equivalent Playwright TypeScript (see test-generation.md):

```
playwright-cli snapshot                          # find refs
playwright-cli fill e3 "John Doe"                # -> page.getByRole('textbox', {
playwright-cli press Enter
playwright-cli click e7
```

For each `- expect:` bullet, add an explicit assertion. See test-generation.md for details.

Collect the generated code and write the test file at the path given in the spec:

```
// spec: specs/basic-operations.plan.md
// seed: tests/seed.spec.ts
import { test, expect } from './fixtures';   // or '@playwright/test' if no fixt

test.describe('Singing in and out', () => {
  test('should sign in', async ({ page }) => {
    // 1. Navigate to the application
    // (handled by the seed fixture)

    // 2. Type 'John Doe' into the username field
    await page.getByRole('textbox', { name: 'username' }).fill('John Doe');

    // 3. Type password
    await page.getByRole('textbox', { name: 'password' }).fill('TestPassword');

    // 4. Press Enter to submit
    await page.getByRole('textbox', { name: 'password' }).press('Enter');

    await expect(page.getByRole('heading')).toContainText('Welcome, John Doe!');
  });
});
```

Rules:

- **One test per file.** File path, describe name, and test name come verbatim from the spec (minus the ordinal).
- Prefix each numbered step with a `// N. <step text>` comment before its actions.
- Use the describe group name verbatim from the spec (no `1.` ordinal).
- Import from `./fixtures` if the project has one; otherwise `@playwright/test`.
- **Important**: close the CLI session and stop the background test before moving to the next scenario.

2.3 Generate multiple scenarios

Loop 2.2 over the targeted scenarios one at a time, restarting the seed between each so every test starts from a clean page. This is safe to parallelise due to unique generated session names - just make sure each test run is stopped.

2.4 Run generated tests

After generation, run the new tests once:

```
PLAYWRIGHT_HTML_OPEN=never npx playwright test tests/<group>/<scenario>.spec.ts
```

Any failure goes to Section 3.

---

3. Heal

Goal: fix failing tests, and update the spec if the app's intended behaviour changed.

3.1 Find failing tests

```
PLAYWRIGHT_HTML_OPEN=never npx playwright test
```

Record the list of failing `<file>:<line>` entries and process them one at a time. Do not attempt parallel fixes — shared state and the single CLI session make that fragile.

3.2 Debug one failure

Run the single failing test in debug mode in the background, then attach:

```
PLAYWRIGHT_HTML_OPEN=never npx playwright test tests/<group>/<scenario>.spec.ts:
# wait for "Debugging Instructions" and the tw-XXXX session name
playwright-cli attach tw-XXXX
```

The test is paused at the start. Step forward or run to until just before the failing action or assertion, then diagnose:

```
playwright-cli snapshot                # did the element change / move / rename?
playwright-cli console                 # app-side errors?
playwright-cli network                 # failed request? wrong payload?
playwright-cli show --annotate         # ask the user to point somewhere
```

Common causes: selector drift, new wrapper element, label/ARIA rename, timing (transition, async load), assertion text updated in the app, test data leaking between runs.

Rehearse the corrected interaction with `playwright-cli` — the generated code in the output is what you paste back into the test.

3.3 Apply the fix

Edit the test file: update the locator, assertion, step order, or inputs to match the corrected behaviour. Stop the background debug run. Rerun the single test to confirm green.

Never skip hooks or add sleeps as a fix. Never use `networkidle`.

3.4 Reconcile with the spec

Open the spec referenced by the `// spec:` header in the test file and locate the scenario that matches the test.

- **Fix was purely technical** (locator drift, better assertion shape) and the spec's user-level behaviour still matches the app → leave the spec alone.
- **Fix changed user-visible steps, inputs, order, or expected outcomes** that the spec describes → update the spec to match reality. Keep the scenario id and file path stable; only the step / expect lines change.
- **Unclear whether the app change is intentional** (spec is stale) **or a regression** (test was right, app is wrong) → **stop and ask the user**. Provide:
  - the scenario id (e.g. `2.3`),
  - the spec lines that no longer match,
  - the observed app behaviour (quote a snapshot excerpt or a concrete outcome).

Only after the user answers, either update the spec (intentional change) or file/flag the test as covering a bug (regression).

3.5 Iteration and giving up

- Fix failures one at a time; rerun after each.
- If after thorough investigation you are confident the test is correct but the app is wrong *and* the user has confirmed it's a bug: mark the test `test.fixme(...)` with a comment pointing at the user's decision or issue link. Never silently skip.

Cross-references

For…	See
`--debug=cli` / attach mechanics	playwright-tests.md
How `playwright-cli` actions become TS	test-generation.md
Mocking requests during exploration/generation	request-mocking.md
Managing the CLI browser session	session-management.md